CPE 486/586: Machine Learning for Engineers

11 Unsupervised Learning: Clustering

Fall 2026

Rahul Bhadani

Outline

1. Motivation
2. Clustering
3. Evaluating Cluster Quality



Motivation

What is Unsupervised Learning?

- In contrary to supervised learning, in unsupervised learning, there is no labeled data.
- That is to say, there is no human-supervision involved in labeling a feature vector.

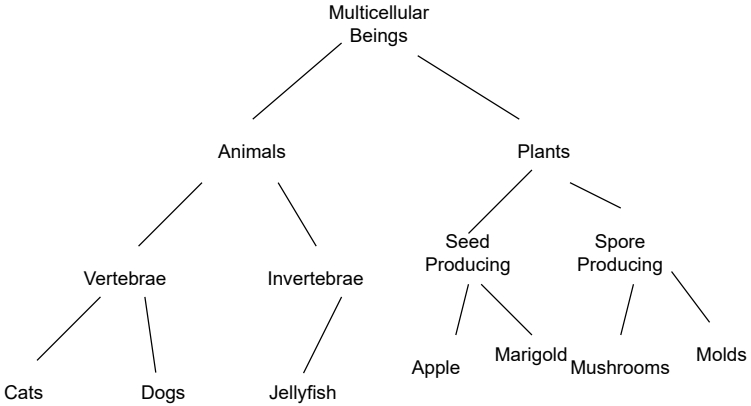
When to use Unsupervised Learning?

1. When we are not sure what we are looking at, unsupervised learning is a useful technique.
2. When we want to just group together several things but we don't care about what these groups belong to.
3. To find a hidden pattern.

When Unsupervised Learning is Useful?

1. Market segmentation: finding relationships between customer variables so that a company can plan its marketing and choose which customer segments to target.
2. Image grouping: splitting a data set of cat and dog photographs into two groups.
3. Image segmentation: dividing an image into regions of similar color or texture.
4. Biological taxonomy (next slide).

Example: Biology Taxonomy





Clustering

Clustering

- Clustering is a type of unsupervised learning aimed at dividing unlabeled data into various groups.
- Clustering places points with similar features in the same group and points with dissimilar features in different groups.
- Clustering appears in market segmentation, outlier detection, network analysis and the applications listed earlier.

Types of Clustering

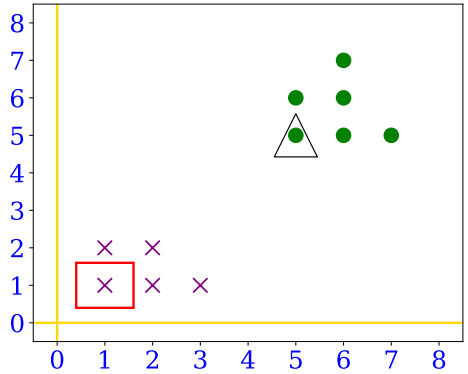
1. Centroid-based or Partition-based clustering
   - Example: K-means
2. Connectivity-based clustering
   - Example: Hierarchical clustering
3. Density-based clustering
   - Example: DBSCAN
4. Graph-based clustering
   - Example: Affinity Propagation
5. Distribution-based Clustering
   - Example: Gaussian Mixture-Models

K-Means

- K-means is the most common clustering method. It is a heuristic that seeks to minimize the within-cluster sum of squared distances to the centroids.
- It starts by assuming the number of clusters, K.
- With K clusters, we start by initializing K centroids.
- Consider the following data points in two clusters:
  - Cluster 1: (1,1), (2,1), (1,2), (2,2), (3,1)
  - Cluster 2: (5,5), (5,6), (6,5), (6,6), (7,5), (6,7)

K-Means Clustering Example

Step 1: Initialize two centroids. Here we pick the data points (1,1) and (5,5).



K-Means Clustering Example (Continued)

- **Step 2:** Calculate Euclidean distance of each point to the centroid and assign each data point to the nearest centroid.
- In this case, it is clear to see that data points closer to (1,1) are (1,1), (2,1), (3,1), (1,2), (2,2) and data points closer to (5,5) are (5,5), (5,6), (6,5), (6,6), (7,5), and (6,7).
- Now, we recalculate two centroids (x_1, y_1) and (x_2, y_2).

$$
\begin{aligned}
x_1 &= \frac{1+2+3+1+2}{5} = \frac{9}{5} = 1.8 \\
y_1 &= \frac{1+1+1+2+2}{5} = \frac{7}{5} = 1.4 \\
x_2 &= \frac{5+5+6+6+7+6}{6} = \frac{35}{6} = 5.83 \\
y_2 &= \frac{5+6+5+6+5+7}{6} = \frac{34}{6} = 5.66
\end{aligned}
\tag{1}
$$

K-Means Iteration

- Hence, the new centroids are (1.8, 1.4) and (5.83, 5.66).
- Next, we reassign the points by computing their distances to the two new centroids.
- If a point is (x_i, y_i) and the two centroids are (x_{c1}, y_{c1}) and (x_{c2}, y_{c2}), then:
  - If $\sqrt{(x_i - x_{c1})^2 + (y_i - y_{c1})^2} < \sqrt{(x_i - x_{c2})^2 + (y_i - y_{c2})^2}$, then (x_i, y_i) belongs to cluster 1.
  - Otherwise, it belongs to cluster 2.
  - With the new centroids, the assignments stay the same, so K-means stops after this pass.

K-Means Convergence

- We repeat the assignment and update steps until the assignments stop changing. The result is a local minimum of the within-cluster sum of squares, so a different initialization can give a different answer.
- For K clusters, point (x_i, y_i) goes to cluster $c_i = \arg\min_j \left((x_i - x_{cj})^2 + (y_i - y_{cj})^2\right)$.

Hierarchical Clustering

- Used to group similar objects.
- We don't need a pre-determined number of clusters.
- There are two types of hierarchical clustering:
  1. Agglomerative clustering
  2. Divisive hierarchical clustering

Agglomerative Clustering

- The algorithm starts with one cluster per data point and repeatedly merges the two closest clusters.
- A linkage rule defines the distance between two clusters: single (closest pair of points), complete (farthest pair), average, or Ward (smallest increase in within-cluster variance).
- Usually agglomerative clustering is presented as a dendrogram.

Agglomerative Clustering (Continued)

- Then we can cut the dendrogram at any point to get the desired number of clusters.

Divisive Hierarchical Clustering

- We start with one cluster that holds the whole data set and split it repeatedly until no cluster holds more than one point.
- The dendrogram shows the merge or split order in two dimensions, whatever the number of features.
- Agglomerative clustering stores an $n \times n$ distance matrix, so it suits modest sample sizes.

Density-based Clustering

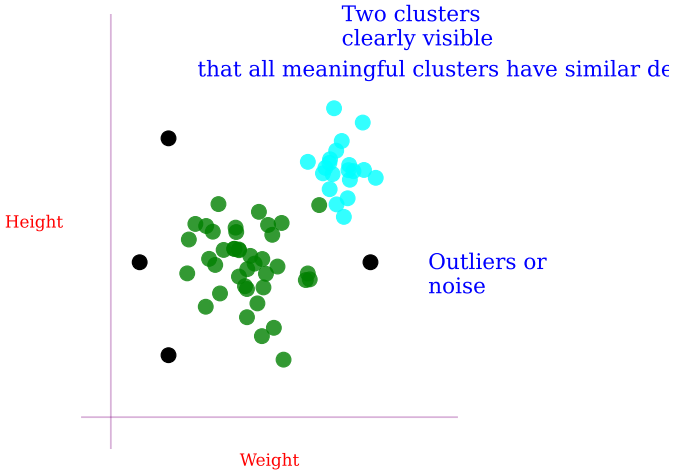
- Density-based methods group points that lie in dense regions. Density is measured by counting the neighbors within a distance ε.
- These methods find regions where points are concentrated and treat sparse or empty regions as gaps between clusters. Points outside the clusters are labeled as noise.

Density-based Spatial Clustering of Applications with Noise (DBSCAN)

- The DBSCAN algorithm groups points that lie in dense regions. Regions of low density separate the clusters.
- DBSCAN uses a specified distance to separate dense clusters from sparser noise. With a spatial index and a small ε, it runs in about $O(n \log n)$ time. Its worst case is $O(n^2)$.

DBSCAN Limitations and Example

- DBSCAN works well only when one search distance ε suits the clusters. That requires the meaningful clusters to have similar densities.
- By eye, the clusters are easy to see. K-means still assigns the outliers to a cluster, and they pull the centroids toward them.



Points to Consider While Applying DBSCAN

- Clusters form high-density regions.
- Noise points are sparse and lie away from the dense regions.

Let’s Start with Unclustered Data

DBSCAN - Step 1: Count Close Points

Step 1: Count the number of points close to each point.

DBSCAN - Step 1: Count Close Points

- We start with the red point and draw an orange circle around it.
- The orange circle contains 8 other points.
- So the red point has 8 neighbors.
- Note: The radius of the circle, ε (eps), is a user-defined parameter.
- Repeating this for another red point, we find 5 neighbors.
- The green point is not close to any other point.
- A core point has at least 4 neighbors within ε. This threshold, minPts, is a second user-defined parameter.

DBSCAN - Defining Core and Non-Core Points

- Points with at least 4 neighbors are core points (red). The remaining points are non-core points (green).

DBSCAN - Building the First Cluster

- We pick a core point at random and assign it to the first cluster.
- Core points within ε of that point join the first cluster.
- Core points within ε of the growing cluster join it in turn.
- When no further core point can join, we add the non-core points that lie within ε of a core point in the cluster. These border points join the cluster without extending it.
- This completes the first cluster.

DBSCAN - Building Subsequent Clusters

- Now we look at other core points that are not part of the first cluster and repeat the process to get the second cluster.
- Non-core points with no core point within ε are labeled noise.

Graph-based Clustering

- Graph-based methods treat the data as a graph, with points as nodes and pairwise similarities as edge weights. They split the graph into groups of strongly connected nodes. Spectral clustering is the standard example.

Affinity Propagation Clustering

- Affinity propagation chooses the number of clusters itself. A user-set preference, placed on the diagonal $S(k, k)$, controls how many exemplars emerge.
- In simple terms, a data point sends messages to the other points. A message informs the target of its relative attractiveness to the sender.
- Each target then responds to all senders with a reply informing each sender of its availability to associate with the sender, given the attractiveness of the messages that it has received from all other senders.

Affinity Propagation Details

- Senders reply to the targets with messages informing each target of the target's revised relative attractiveness to the sender, given the availability messages it has received from all the targets.
- The message-passing procedure is repeated until a consensus is achieved.
- Once the sender is associated with one of its targets, that target becomes the point's exemplar.
- All points with the same exemplar are placed in the same cluster.
- Note: The similarity $S(i, k)$ is the input to the algorithm. A common choice is the negative squared Euclidean distance. The attractiveness messages above are the responsibilities $R(i, k)$.

Affinity Propagation Example

- Example: (1,2) and (3,4)
- $-\left[(1-3)^2+(2-4)^2\right]=-8$ is the similarity score between (1,2) and (3,4).

Affinity Propagation Algorithm

1. Calculate the similarity matrix S over pairs of points, say the negative squared Euclidean distance. Set the diagonal $S(k, k)$ to the preference, for example the median of the similarities.

	$A1$	$A2$	$A3$	$\cdots$
$A1$	□	□	□	□
$A2$	□	□	□	□
$A3$	□	□	□	□
$\vdots$	□	□	□	□

Affinity Propagation Algorithm

2. Calculate the Responsibility Matrix:
   - Initialize the availability matrix A and the responsibility matrix R with zeros.
   - Update R iteratively using the formula:

$$R(i,k) = S(i,k) - \max_{k' \neq k} \{A(i,k') + S(i,k')\}$$

Affinity Propagation Algorithm

3. Then, we calculate the availability matrix. For $i \neq k$:

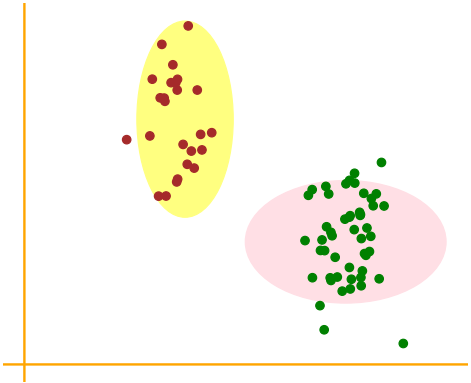
$$A(i,k) = \min\left(0, R(k,k) + \sum_{\substack{i' \neq i \\ i' \neq k}} \max(0, R(i',k))\right)$$

and on the diagonal:

$$A(k,k) = \sum_{i' \neq k} \max(0, R(i',k))$$

4. Exemplars (Cluster Centers): point k is an exemplar when $A(k,k) + R(k,k) > 0$.
5. Once we have exemplars, assign a data point to its most similar exemplar. Points that share an exemplar form one cluster.

Distribution-based Clustering



- Distribution-based methods group data points together based on their likelihood of belonging to the same probability distribution.

Distribution-based Clustering - Notes

- Distribution-based methods use statistical inference to cluster data. The closer a point lies to a component's center, the higher its probability of assignment to that component.
- Distribution-based methods can fit cluster shapes, such as elongated ellipses, that K-means misses. This works when the clusters follow the assumed distribution.
- A Gaussian mixture overfits when it has too many components, or full covariance matrices with too little data. A component can also collapse onto a single point, which drives its variance to zero and the likelihood to infinity.

Gaussian Mixture Model (GMM)

- GMM assumes the data come from a mixture of K Gaussian distributions with unknown parameters. A point is drawn from one component, chosen with probability equal to that component's mixing weight. The Expectation-Maximization (EM) algorithm fits the parameters.
- The parameters start at random values. In the E-step, the current parameters give the probability that a point belongs to a given component.
- In the M-step, the component parameters are re-estimated from the data, with points weighted by those probabilities.

Gaussian Mixture Model - E-Step

1. E-Step:
   1. Compute the probability that each data point belongs to each Gaussian component.
   2. Then soft-assign a point to clusters, that is, a point $\mathbf{x}$ belongs to cluster 1 with 60% probability, to cluster 2 with 30% probability and to cluster 3 with 10% probability (just an example).
   3. The probability that point $\mathbf{x}_i$ belongs to component k is

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

Gaussian Mixture Model - M-Step

2. M-Step:
    1. Update the parameters of each Gaussian component based on the soft assignments.
    2. Recalculate parameters of Gaussian components such as mean, variance, and weights.
3. The two steps repeat until the log-likelihood rises by less than a set tolerance.



Evaluating Cluster Quality

Evaluating Cluster Quality – Extrinsic Measure

1. Extrinsic Measure: Requires ground truth labels.
    1. Rand Index
    2. Mutual Information
    3. V-measure
    4. Fowlkes-Mallows Scores

Evaluating Cluster Quality - Rand Index

1. Rand Index:
   - Measures the similarity between the predicted and true cluster assignments through pairwise comparisons. A pair counts as correct when both assignments place it in the same cluster, or both place it in different clusters.
   - Higher Rand Index means higher similarity.
   - $$RI = \frac{\text{\# pairwise correct predictions}}{\text{Total \# possible pairs}}$$
   - The Adjusted Rand Index corrects for agreement between two clusterings that occurs by chance.

Evaluating Cluster Quality - Mutual Information, V-measure, Fowlkes-Mallows Scores

2. Mutual Information: The degree of agreement between clusters is computed by joint and marginal probabilities.
3. V-measure: Measures the correctness of cluster assignments using conditional entropy analysis.
4. Fowlkes-Mallows Scores: Measures the correctness of the cluster assignments using pairwise precision and recall. A higher score signifies higher correctness.

Evaluating Cluster Quality – Intrinsic Measures

2. Intrinsic Measure: Doesn't require ground truth labels.
    1. Silhouette Coefficients: Measures the between-cluster distance against within-cluster distance. A higher score signifies better-defined clusters.
    2. Calinski-Harabasz Index: Measures the between-cluster dispersion against within-cluster dispersion.
    3. Davies-Bouldin Index: Measures the size of clusters against the average distance between clusters. A lower score signifies better-defined clusters.

Cluster Quality Evaluation Formulas

1. ARI (Adjusted Rand Index)

- Expected index $= \frac{\text{\# pairs in the same cluster (actual)} \times \text{\# pairs in the same cluster (predicted)}}{\text{Total no. of possible pairs}}$
- $ARI = \frac{\text{\# pairs in the same cluster in both} - \text{Expected index}}{\text{Average \# pairs in same cluster for actual and predicted} - \text{Expected index}}$

Cluster Quality Evaluation Formulas (Continued)

2. Mutual Information (MI)

- Adjusted MI (AMI)
- Normalized MI (NMI)
- $P(i,j) =$ Probability of data in cluster i (actual) and cluster j (predicted)
- $P(i) =$ Probability of data in cluster i (actual)
- $P'(j) =$ Probability of data in cluster j (predicted)
- $H(U) =$ Entropy of actual cluster assignment $= -\sum_{i \in U} P(i) \log P(i)$, where U is the set of actual clusters
- $H(V) = -\sum_{j \in V} P'(j) \log P'(j)$, where V is the set of predicted clusters
- $MI = \sum_{i \in U} \sum_{j \in V} P(i,j) \log \frac{P(i,j)}{P(i)P'(j)}$
- $NMI = \frac{MI}{\frac{1}{2}[H(U)+H(V)]}$
- $AMI = \frac{MI - E[MI]}{\frac{1}{2}[H(U)+H(V)] - E[MI]}$

Cluster Quality Evaluation Formulas (Continued)

3. V-Measure

- $H(C|K) =$ conditional entropy of the true classes C given the cluster assignments K
- $H(C|K) = -\sum_c \sum_k \frac{n_{ck}}{n} \log\left(\frac{n_{ck}}{n_k}\right)$, where $\frac{n_{ck}}{n_k} = \frac{\text{no. of samples labelled } c \text{ in cluster } k}{\text{Total no. of samples in cluster } k}$
- $H(C) = -\sum_c \frac{n_c}{n} \log\left(\frac{n_c}{n}\right)$
- $\text{Homogeneity}(h) = 1 - \frac{H(C|K)}{H(C)}$
- $H(C|K) = -\sum_k n_{ck} \log\left(\frac{n_{ck}}{n_k}\right) \rightarrow \frac{\text{no. of samples labelled C in cluster k}}{\text{Total no. of samples in cluster k}}$
- $\text{Completeness}(c) = 1 - \frac{H(K|C)}{H(K)}$
- $V \text{ measure} = \frac{2 \cdot h \cdot c}{h + c}$

Cluster Quality Evaluation Formulas (Continued)

- **4. Fowlkes-Mallows Scores**
  - $FMI = \frac{TP}{\sqrt{(TP+FP) \times (TP+FN)}}$
  - $TP \rightarrow$ pairs in the same cluster in both the true and predicted assignments
  - $FP \rightarrow$ pairs in the same cluster in the prediction only
  - $FN \rightarrow$ pairs in the same cluster in the truth only
- **5. Silhouette Coefficient (S)**
  - $a =$ Average distance between sample and all other points in the same cluster
  - $b =$ Average distance between sample and the points in the nearest other cluster
  - $s = \frac{b-a}{\max(a,b)}$ for one sample. The silhouette coefficient S is the mean of s over the samples.

Cluster Quality Evaluation Formulas (Continued)

6. Calinski-Harabasz Index

- $K =$ no. of clusters
- $n_q =$ no. of points in cluster q
- $C_q =$ cluster center of cluster q
- $n_E =$ no. of data points
- $C_E =$ cluster center of all points
- Between-cluster dispersion, $B = \sum_{q=1}^{K} n_q (C_q - C_E)(C_q - C_E)^T$
- Within-cluster dispersion, $W = \sum_{q=1}^{K} \sum_{x \in \text{cluster } q} (x - C_q)(x - C_q)^T$
- Calinski-Harabasz Score $(s) = \frac{\text{tr}(B)}{\text{tr}(W)} \times \frac{n_E - K}{K - 1}$

Cluster Quality Evaluation Formulas (Continued)

7. Davies-Bouldin Index

- $K =$ # clusters
- $S_i =$ Average distance between each point in cluster i to cluster center C_i
- $d_{ij} =$ distance between cluster centers C_i and C_j
- Similarity measure $R_{ij} = \frac{\text{Sum of within-cluster distances}}{\text{Between-cluster distance}} = \frac{S_i + S_j}{d_{ij}}$
- $DB =$ Average of maximum similarity measures $= \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i}(R_{ij})$

Python Notebook

https://github.com/rahulbhadani/CPE486586_FA26/blob/master/Code/CPE486586_Ch11_Unsupervised_Learning.ipynb

The End